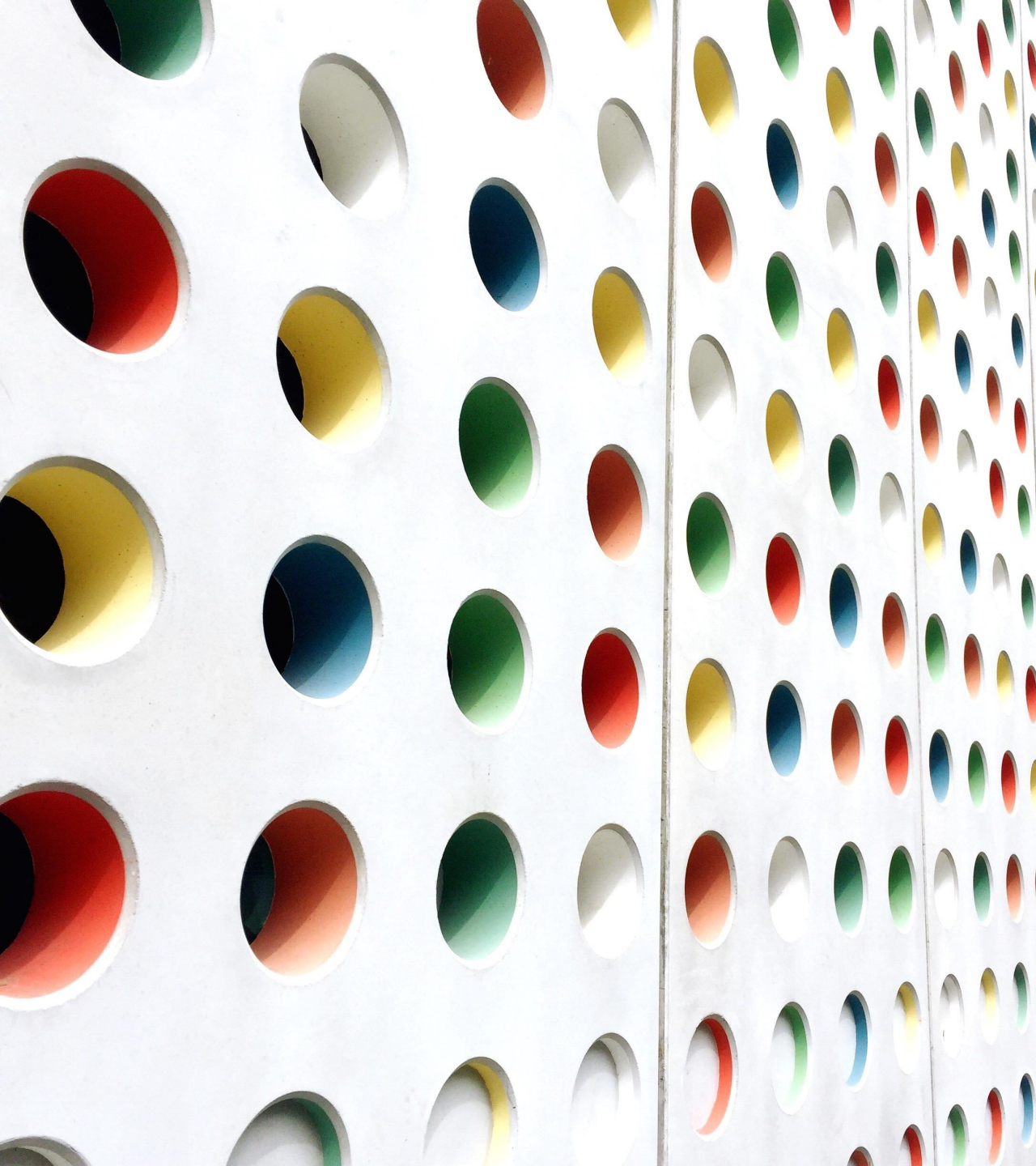




# Graph

---

M. ASAD ARSHED



# Graph

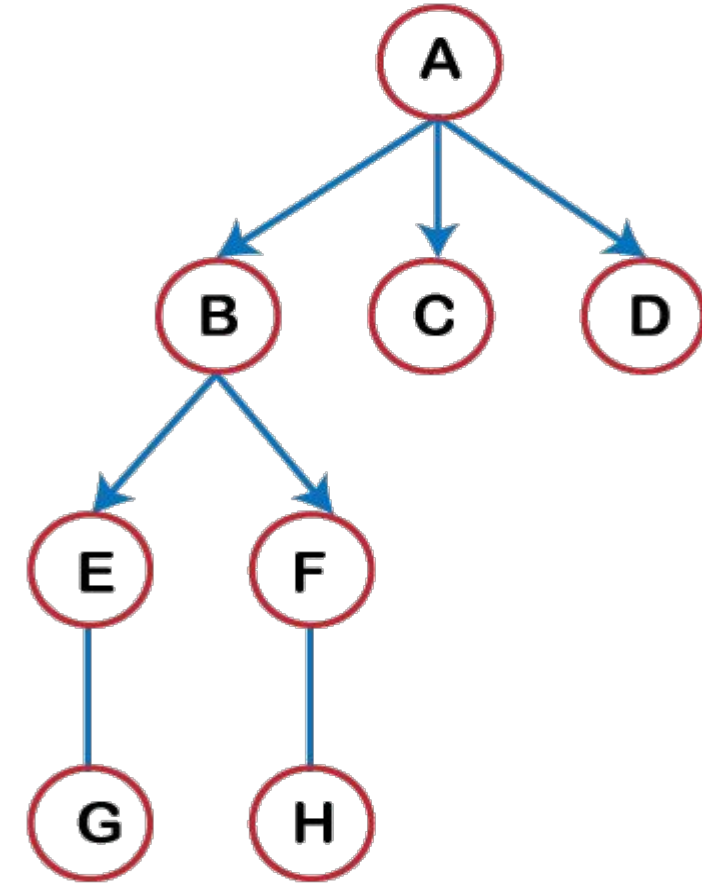
---

A graph in C++ is a non-linear data structure defined as a collection of vertices and edges. Linear data structure is a structure in which all the elements are stored sequentially and have only single level. In contrast, a non-linear data structure is a structure that follows a hierarchy, i.e., elements are arranged in multiple levels.

# What is Non-Linear?

---

In the above figure, we can assume the company hierarchy where A represents the CEO of the company, B, C and D represent the managers of the company, E and F represent the team leaders, and G and H represent the team members. This type of structure has more than one level, so it is known as a non-linear data structure.



# Graph vs Tree

## The basis of Comparison

### Definition

#### Graph

Graph is a non-linear data structure.

#### Tree

Tree is a non-linear data structure.

### Structure

It is a collection of vertices/nodes and edges.

It is a collection of nodes and edges.

### Edges

Each node can have any number of edges.

If there is  $n$  nodes then there would be  $n-1$  number of edges

### Types of Edges

They can be directed or undirected

They are always directed

### Root node

There is no unique node called root in graph.

There is a unique node called root(parent) node in trees.

### Loop Formation

A cycle can be formed.

There will not be any cycle.

### Traversal

For graph traversal, we use Breadth-First Search (BFS), and Depth-First Search (DFS).

We traverse a tree using in-order, pre-order, or post-order traversal methods.

### Applications

For finding shortest path in networking graph is used.

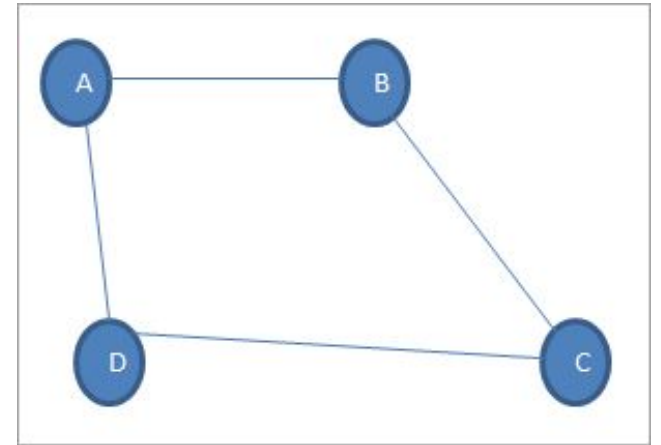
For game trees, decision trees, the tree is used.

# Terminologies

---

**Vertex:** Each node of the graph is called a vertex. In the above graph, A, B, C, and D are the vertices of the graph.

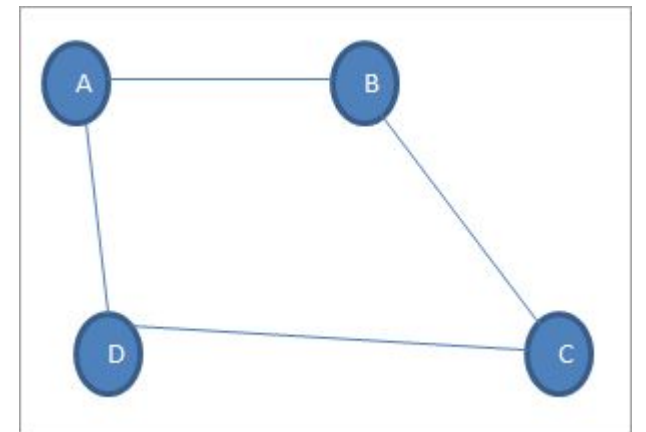
**Edge:** The link or path between two vertices is called an edge. It connects two or more vertices. The different edges in the above graph are AB, BC, AD, and DC.



# Terminologies

---

**Adjacent node:** In a graph, if two nodes are connected by an edge then they are called adjacent nodes or neighbors. In the above graph, vertices A and B are connected by edge AB. Thus A and B are adjacent nodes.



# Terminologies

---

**Closed path:** If the initial node is the same as a terminal node, then that path is termed as the closed path.

**Simple path:** A closed path in which all the other nodes are distinct is called a simple path.

**Cycle:** A path in which there are no repeated edges or vertices and the first and last vertices are the same is called a cycle. In the above graph,  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$  is a cycle.

# Terminologies

---

**Connected Graph:** A connected graph is the one in which there is a path between each of the vertices. This means that there is not a single vertex which is isolated or without a connecting edge. The graph shown above is a connected graph.

**Complete Graph:** A graph in which each node is connected to another is called the Complete graph. If  $N$  is the total number of nodes in a graph then the complete graph contains  $N(N-1)/2$  number of edges.



# Terminologies

---

**Weighted graph:** A positive value assigned to each edge indicating its length (distance between the vertices connected by an edge) is called weight. The graph containing weighted edges is called a weighted graph. The weight of an edge  $e$  is denoted by  $w(e)$  and it indicates the cost of traversing an edge.

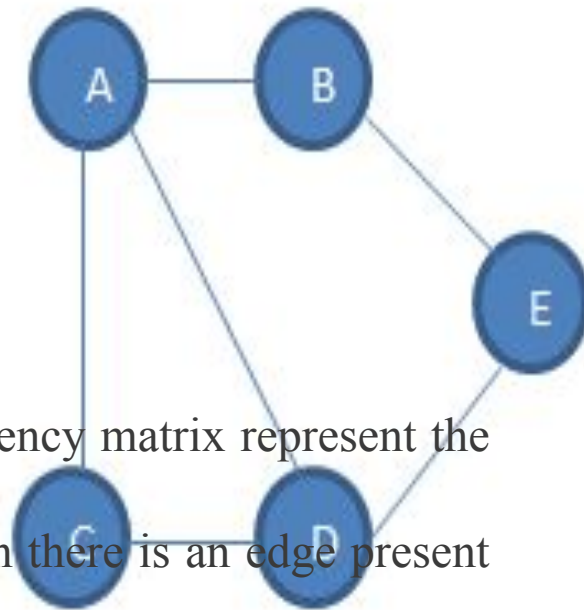
**Diagraph:** A digraph is a graph in which every edge is associated with a specific direction and the traversal can be done in specified direction only.

# Graph Representation

The way in which graphs are represented is called their “representation”. There are two main representations or as

## 1. Sequential Representation

- The rows and columns of the adjacency matrix represent the vertices in a graph.
- The matrix element is set to 1 when there is an edge present between the vertices.
- If the edge is not present then the element is set to 0.



Undirected Graph

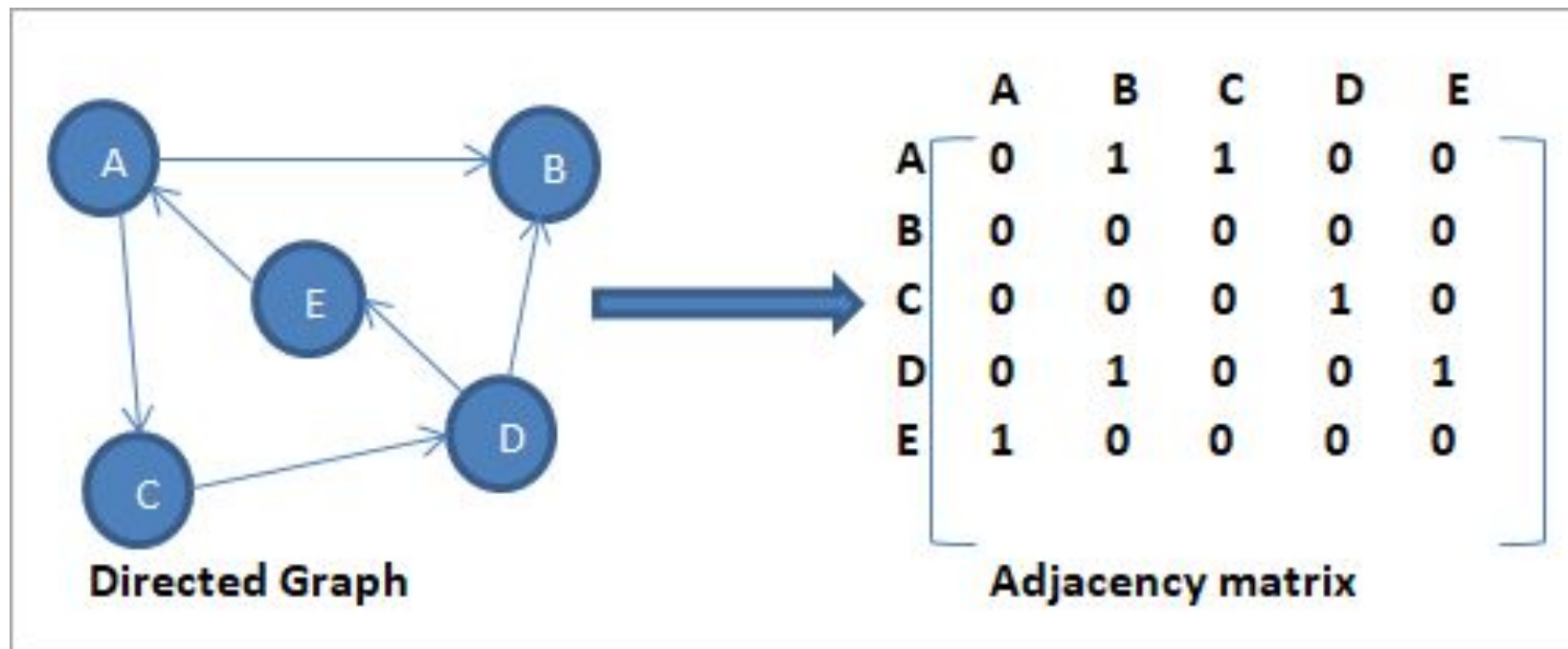


|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 |
| B | 1 | 0 | 0 | 0 | 1 |
| C | 1 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 1 | 0 | 1 |
| E | 0 | 1 | 0 | 1 | 0 |

Adjacency matrix

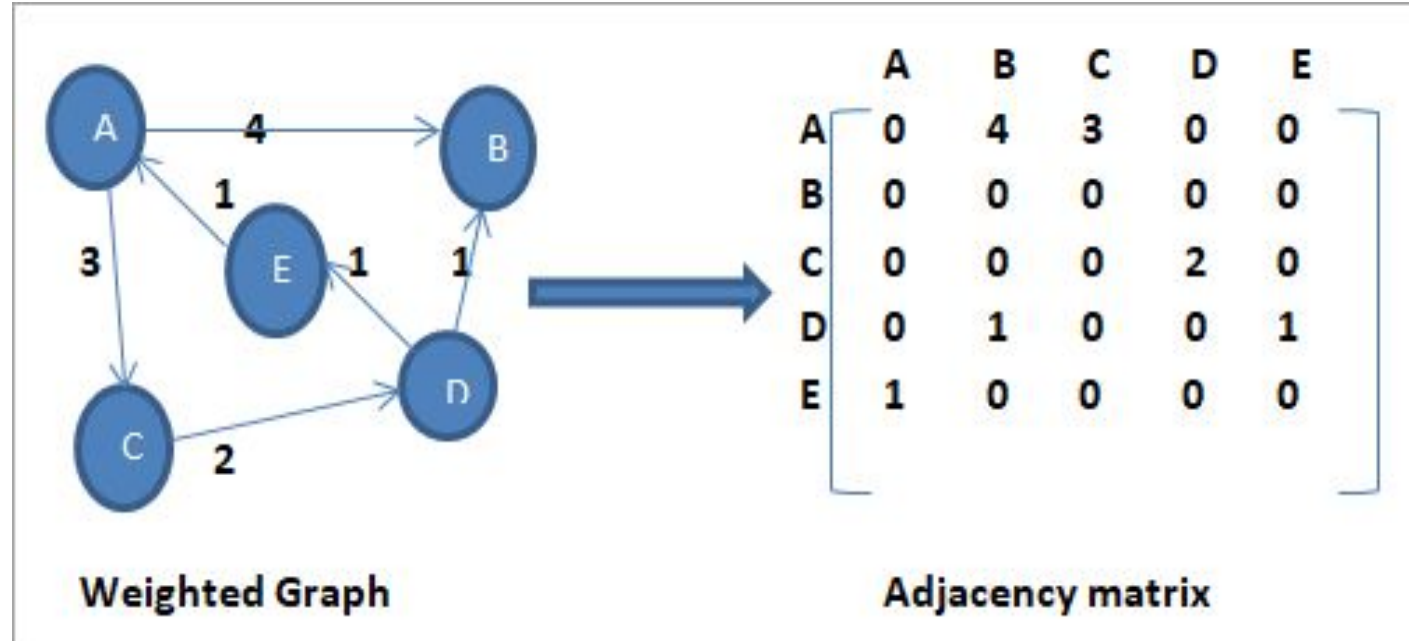
# Graph Representation

Adjacency matrix of a Directed Graph.

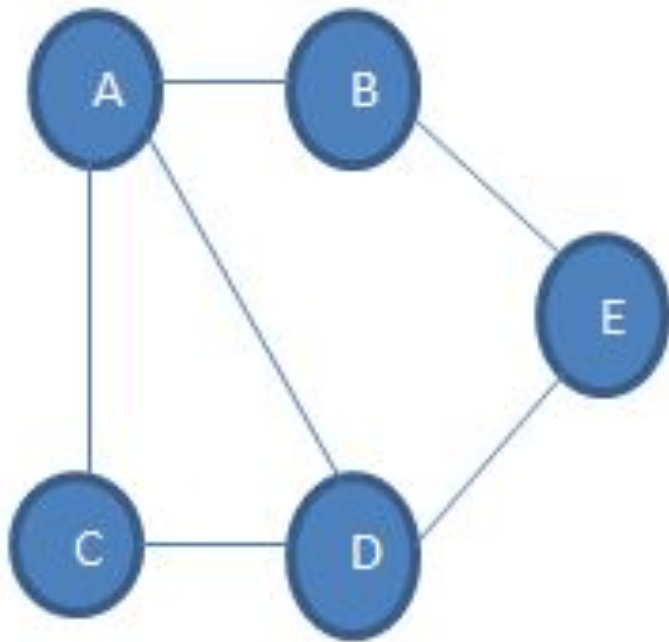


# Graph Representation

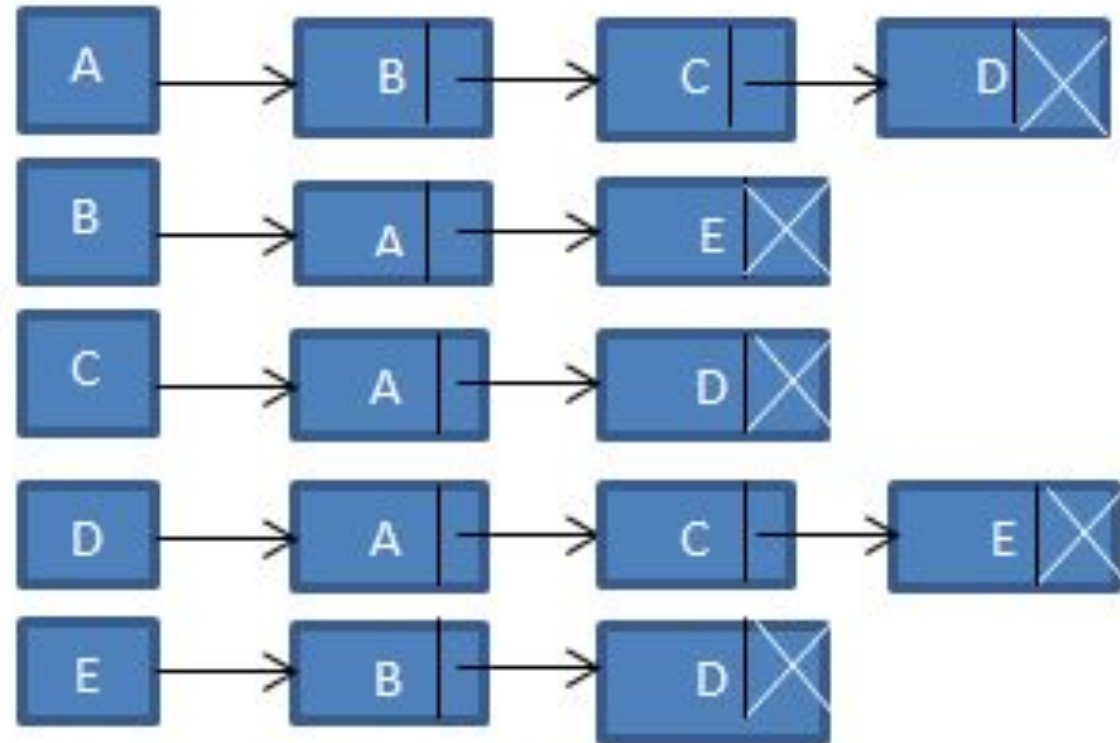
weighted graph and its corresponding adjacency matrix.



# Graph Representation

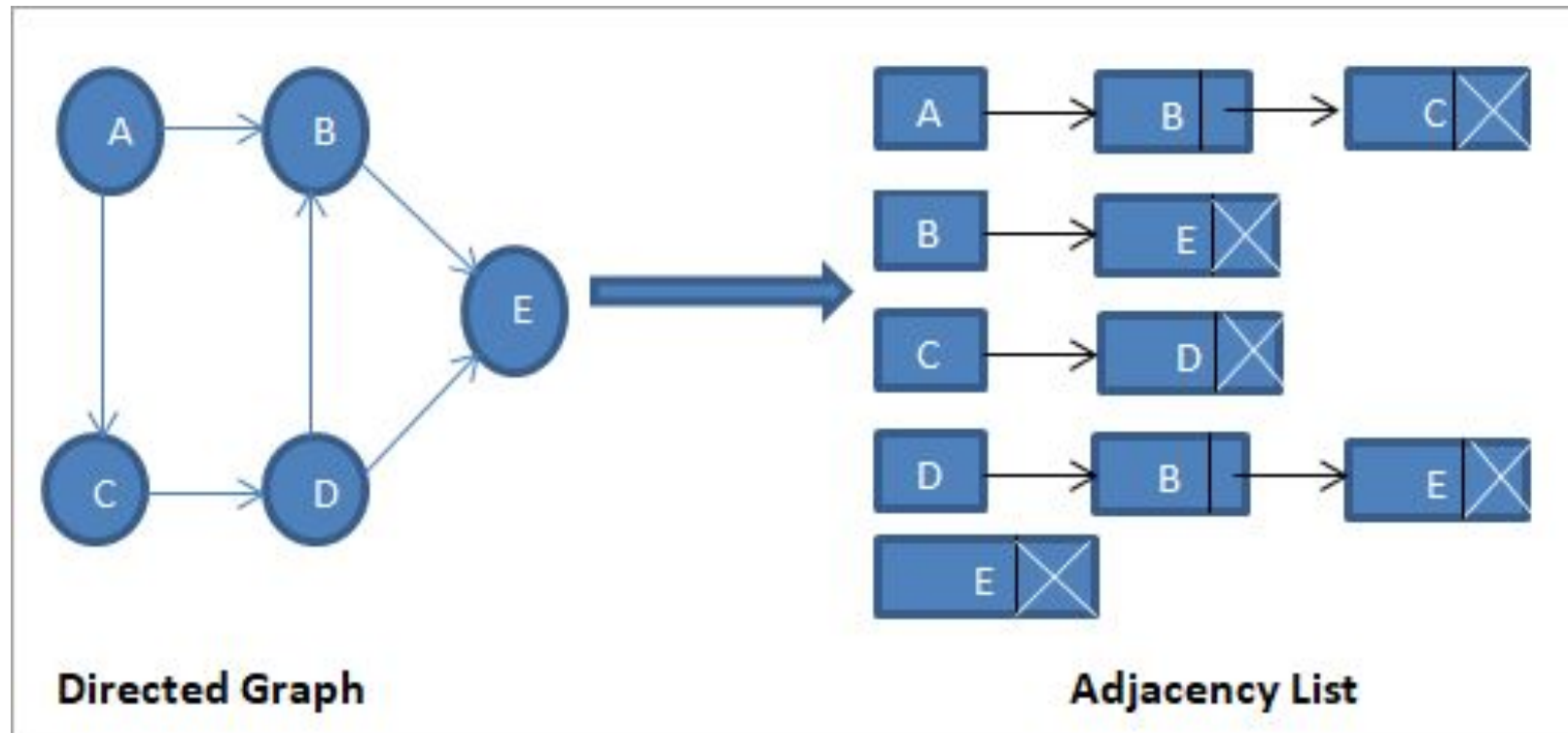


**Undirected Graph**

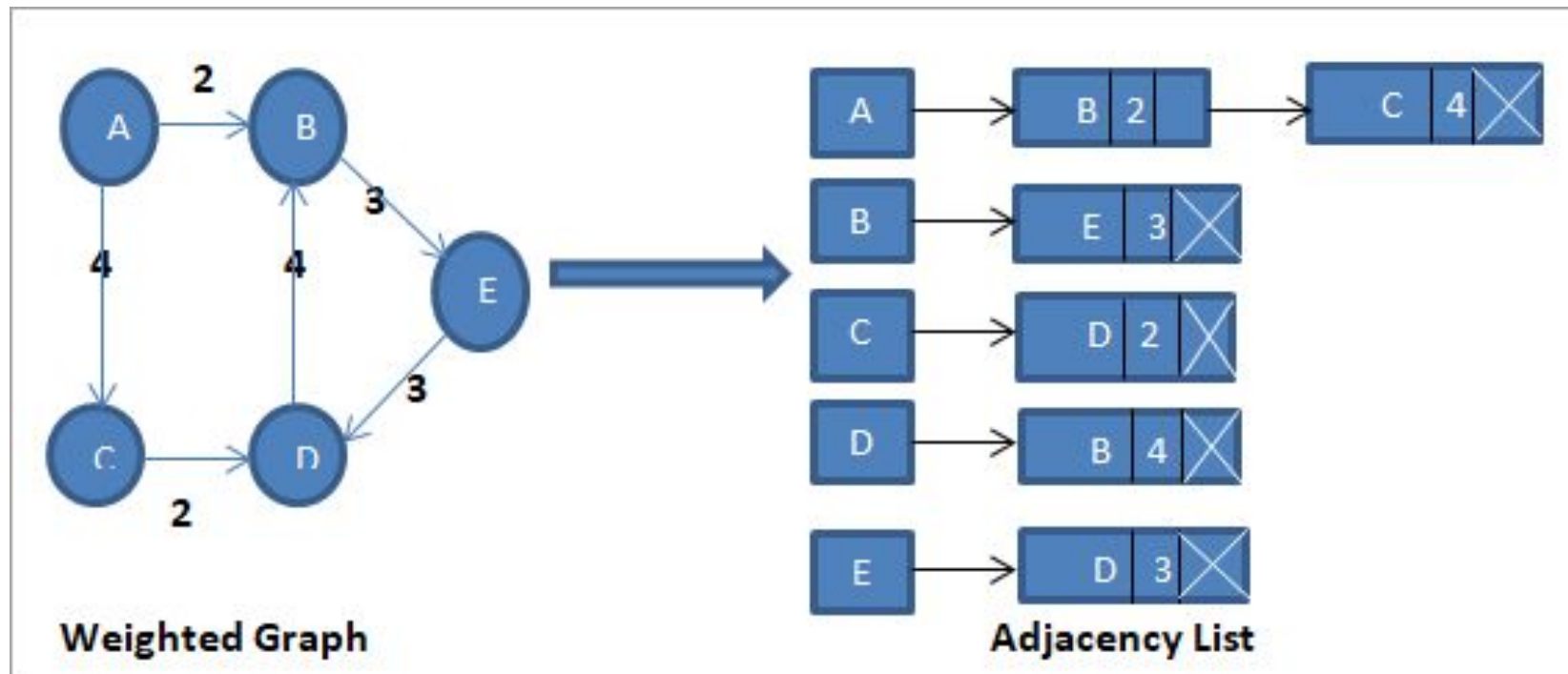


**Adjacency List**

# Graph Representation



# Graph Representation



# Basic Operations

---

**Add a vertex:** Adds vertex to the graph.

**Add an edge:** Adds an edge between the two vertices of a graph.

**Display the graph vertices:** Display the vertices of a graph (Traversing)